

Appunti di Calcolatori - Introduzione

Trascrizione dalle fonti del corso

Indice

1	Introduzione ai Calcolatori	4
1.1	Descrizione del Corso	4
1.2	Struttura e Modalità d'Esame	4
1.3	Evoluzione Storica: dall'ENIAC ad oggi	4
1.4	Perché studiare l'architettura?	4
1.5	Software di Sistema e Linguaggi	4
1.6	Componenti del Calcolatore	5
2	Aritmetica dei Calcolatori	5
2.1	La Rappresentazione Digitale	5
2.2	Sistemi di Numerazione Posizionali	5
2.3	Conversioni di Base	6
2.4	Operazioni Aritmetiche e Shifting	6
2.5	Rappresentazione dei Numeri Interi	6
2.6	Overflow	6
3	Rappresentazione dei Numeri Reali	6
3.1	Limiti della Rappresentazione	6
3.2	Rappresentazione in Virgola Fissa (Fixed Point)	7
3.3	Virgola Mobile (Standard IEEE 754)	7
3.4	Formati e Precisione	7
3.5	Valori Speciali	7
4	Codifica del Testo	8
4.1	Dal Carattere al Bit	8
4.2	Lo Standard ASCII	8
4.3	Unicode e UTF-8	8
4.4	Esempio di Codifica	8
5	Cenni alle Reti Logiche	8
5.1	Valori Logici ed Elettronica	8
5.2	Tipologie di Reti Logiche	9
5.3	Algebra di Boole e Porte Logiche	9
5.4	Circuiti Combinatori Fondamentali	9
5.5	Elementi di Memoria	9

6	Il Linguaggio Assembly	10
6.1	Il Gap tra Umano e Calcolatore	10
6.2	Instruction Set Architecture (ISA)	10
6.3	Operandi e Registri	10
6.4	Controllo del Flusso e Salti	11
6.5	Principi di Progettazione dell'ISA	11
7	L'Architettura RISC-V	11
7.1	Caratteristiche Generali	11
7.2	I Registri nel RISC-V	11
7.3	Organizzazione della Memoria	12
7.4	Trasferimento Dati: Load e Store	12
7.5	Rappresentazione delle Istruzioni	12
7.6	Pseudo-istruzioni	12
8	Flusso di Controllo e Procedure	13
8.1	Istruzioni di Salto Condizionato	13
8.2	Protocollo di Chiamata delle Procedure	13
8.3	Gestione dello Stack e Record di Attivazione	13
8.4	Funzioni Foglia e Non Foglia	14
8.5	Organizzazione della Memoria	14
9	L'Architettura Intel x86	14
9.1	Evoluzione e Caratteristiche CISC	14
9.2	I Registri Intel	14
9.3	Formato delle Istruzioni e Operandi	15
9.4	Modalità di Indirizzamento	15
9.5	Istruzioni Particolari	15
9.6	Convenzione di Chiamata (ABI)	15
10	L'Architettura Intel x86	16
10.1	Evoluzione e Caratteristiche CISC	16
10.2	I Registri Intel	16
10.3	Formato delle Istruzioni e Operandi	16
10.4	Modalità di Indirizzamento	17
10.5	Istruzioni Particolari	17
10.6	Convenzione di Chiamata (ABI)	17
11	L'Architettura ARM	17
11.1	Origini e Caratteristiche	17
11.2	I Registri ARM	17
11.3	Formato delle Istruzioni e Flessibilità	18
11.4	Esecuzione Condizionale	18
11.5	Modalità di Indirizzamento e Write-back	18
11.6	Istruzioni Multiple (LDM e STM)	18

12 La Toolchain: dalla Sorgente all'Esecuibile	19
12.1 Il Processo di Traduzione	19
12.2 Il Ruolo dell'Assembler	19
12.3 Struttura del File Oggetto	19
12.4 Il Linker e la Creazione dell'Esecuibile	20
12.5 Linking Statico e Dinamico	20

1 Introduzione ai Calcolatori

1.1 Descrizione del Corso

Il corso, noto anche come **Architettura degli elaboratori**, si concentra principalmente su lezioni teoriche integrate da esercitazioni pratiche riguardanti l'aritmetica dei calcolatori e il linguaggio **Assembly** [1]. In totale, l'insegnamento prevede **48 ore** di didattica frontale distribuite in 12 settimane [1].

1.2 Struttura e Modalità d'Esame

Il programma è suddiviso in tre parti principali:

1. **Prima parte:** Introduzione, aritmetica dei calcolatori e cenni sulle reti logiche [1].
2. **Seconda parte:** Linguaggio Assembly, con focus su architetture RISC-V, INTEL e ARM [1].
3. **Terza parte:** Elementi di architettura dei calcolatori, inclusi CPU, Pipeline e gerarchie di memoria [1].

L'esame prevede una **prova scritta**, spesso svolta su piattaforma Moodle con quesiti a risposta multipla, e un **orale opzionale** [2]. Quest'ultimo è obbligatorio per voti compresi tra 15 e 17 o per confermare voti alti come la lode [2]. Il libro di riferimento è "*Struttura e progetto dei calcolatori - Progettare con RISC-V*" di Patterson e Hennessy [2].

1.3 Evoluzione Storica: dall'ENIAC ad oggi

I calcolatori elettronici rappresentano oggi una tecnologia vitale che produce il 10% del PIL degli Stati Uniti [2].

- **ENIAC (1946):** Considerato uno dei primi calcolatori, fu commissionato per calcolare le traiettorie dei proiettili; occupava una stanza di 9x30 metri e consumava enormi quantità di energia [2].
- **La rivoluzione:** Attualmente i calcolatori pervadono ogni aspetto della vita, dalle automobili ai telefoni cellulari, fino alla robotica e alla mappatura del genoma [2].

1.4 Perché studiare l'architettura?

È essenziale per un programmatore conoscere l'organizzazione del calcolatore per scrivere software efficiente, comprendendo come sfruttare la **gerarchia di memoria** e il **parallelismo** [3]. Le prestazioni dipendono dall'interazione tra algoritmi, compilatore, hardware e l'**ISA** (Instruction Set Architecture), che definisce le istruzioni riconosciute dalla CPU [3, 4].

1.5 Software di Sistema e Linguaggi

Il software di sistema agisce come intermediario tra l'utente e l'hardware:

- **Sistema Operativo (SO):** Gestisce l'input/output, la memoria e il multitasking [2].

- **Compilatore:** Traduce il codice ad alto livello (come il C) in linguaggio macchina [4].

L'**Assembly** introduce codici mnemonici per rendere le istruzioni macchina gestibili dall'uomo prima della traduzione finale in bit [3, 4].

1.6 Componenti del Calcolatore

Un calcolatore è composto da quattro elementi principali:

1. **Input:** Dispositivi per inviare dati, come tastiera e mouse [5, 6].
2. **Output:** Dispositivi per visualizzare i risultati, come schermi e stampanti [5, 6].
3. **Processore:** Esegue le elaborazioni, diviso in **datapath** (parte operativa) e **unità di controllo** [5, 6].
4. **Memoria:** Unità dove vengono memorizzati dati e programmi in esecuzione [5, 6].

2 Aritmetica dei Calcolatori

2.1 La Rappresentazione Digitale

Un computer è costituito da circuiti elettronici in cui l'informazione è manipolata esclusivamente come sequenze di bit, dove il valore **1** indica il passaggio di corrente e lo **0** l'assenza [1, 2].

- **Bit:** L'unità minima di informazione (0 o 1) [1, 3].
- **Byte:** Una sequenza raggruppata di 8 bit [1, 3].
- **Codifica:** Funzione che associa un simbolo a una sequenza di bit. Una codifica su n bit permette di rappresentare 2^n simboli distinti [1, 4].

2.2 Sistemi di Numerazione Posizionali

I calcolatori utilizzano sistemi posizionali in cui il valore di una cifra dipende dalla sua posizione [5, 6]. Il valore di una sequenza di cifre in base B è dato dalla formula:

$$\sum_{k=0}^i c_k \cdot B^k$$

dove c_k è la cifra in posizione k [5, 7]. Le basi principali sono:

- **Base 10 (Decimale):** Utilizza le cifre da 0 a 9 [5, 6].
- **Base 2 (Binaria):** Base nativa dei circuiti, utilizza solo 0 e 1 [5, 7, 8].
- **Base 16 (Esadecimale):** Utilizza 16 cifre (0-9 e A-F) per rappresentare in modo compatto lunghe sequenze binarie (un byte corrisponde a due cifre esadecimali) [5, 7, 9].

2.3 Conversioni di Base

- **Da Binario a Decimale:** Si sommano le potenze di 2 corrispondenti ai bit impostati a 1 [7, 10].
- **Da Decimale a Binario:** Si utilizza il metodo delle **divisioni successive** per 2, dove i resti (letti dall'ultimo al primo) formano il numero binario [9-11].

2.4 Operazioni Aritmetiche e Shifting

Le operazioni di somma e sottrazione binaria seguono regole simili al sistema decimale, gestendo riporti e prestiti [10-12]. L'operazione di **shifting** consiste nello spostare i bit: aggiungere uno 0 a destra equivale a moltiplicare per 2, mentre traslare i bit a destra eliminando l'ultima cifra equivale a dividere per 2 [10, 13]. I compilatori usano lo shift per ottimizzare i calcoli [10, 13].

2.5 Rappresentazione dei Numeri Interi

Per gestire i numeri negativi si utilizzano diverse tecniche:

- **Modulo e Segno:** Il bit più significativo indica il segno (0 per +, 1 per -). Presenta l'inconveniente della doppia rappresentazione dello zero [14, 15].
- **Complemento a 1 (CA1):** I numeri negativi si ottengono invertendo tutti i bit del positivo. Anche qui lo zero ha due codifiche [14, 16].
- **Complemento a 2 (CA2):** È lo standard moderno [17]. Un numero negativo si ottiene facendo il complemento a 1 e sommando 1 [17, 18].
 - **Vantaggi:** Codifica dello zero unica e sottrazioni eseguibili come somme ($x - y = x + \text{comp}2(y)$) [17, 19].
 - **Intervallo:** Con k bit si rappresentano numeri da -2^{k-1} a $2^{k-1} - 1$ [17, 19].

2.6 Overflow

L'overflow si verifica quando il risultato di un'operazione non è rappresentabile con i bit a disposizione [17, 20]. In aritmetica CA2, l'overflow può avvenire solo se si sommano due numeri con lo **stesso segno** e il risultato presenta il segno opposto [17, 21, 22].

3 Rappresentazione dei Numeri Reali

3.1 Limiti della Rappresentazione

Rappresentare esattamente un numero reale $x \in \mathbb{R}$ non è sempre possibile all'interno di un calcolatore [1]. Questo limite riguarda in particolare i numeri irrazionali (come π) o quelli con una sequenza infinita di cifre decimali non periodiche, che non possono essere contenuti in un numero finito di bit [1, 2]. Per tali ragioni, i calcolatori operano spesso con approssimazioni dei valori reali [1, 2].

3.2 Rappresentazione in Virgola Fissa (Fixed Point)

In questa codifica, su un totale di k cifre, una parte fissa f viene dedicata alla parte frazionaria e la restante parte $k - f$ alla parte intera [3].

- **Vantaggi:** La semplicità delle operazioni aritmetiche, che vengono eseguite come se fossero numeri interi per poi essere riscalate, semplificando il progetto della CPU [4]. Se il numero è rappresentabile, non vengono introdotti errori di approssimazione [4].
- **Svantaggi:** Diventa inefficiente quando l'applicazione deve gestire contemporaneamente numeri molto grandi e numeri molto piccoli (ordini di grandezza diversi) [4, 5].

3.3 Virgola Mobile (Standard IEEE 754)

La **Virgola Mobile** (Floating Point) risolve i limiti della virgola fissa esprimendo il numero nella forma $x = M \cdot B^E$, dove M è la mantissa ed E l'esponente [6]. Lo standard universale è l'**IEEE 754**, che definisce tre componenti principali [7]:

1. **Segno (s):** 1 bit (0 per positivo, 1 per negativo) [7].
2. **Esponente (E):** Memorizzato in notazione "eccesso" (bias) per permettere la gestione di esponenti negativi [7, 8].
3. **Mantissa (M):** Rappresenta la parte frazionaria; nei numeri normalizzati si assume la presenza di un "1" implicito a sinistra della virgola [7, 8].

3.4 Formati e Precisione

I calcolatori utilizzano principalmente due formati [9, 10]:

- **Precisione Singola (float):** 32 bit totali (1 segno, 8 esponente, 23 mantissa). Il bias è 127 [9, 10].
- **Precisione Doppia (double):** 64 bit totali (1 segno, 11 esponente, 52 mantissa). Il bias è 1023 [9, 10].

3.5 Valori Speciali

Lo standard IEEE 754 definisce configurazioni specifiche per gestire eccezioni o limiti hardware [9, 11]:

- **Zero:** Esponente 0 e mantissa 0. Esiste sia lo zero positivo che quello negativo [9, 12].
- **Infinito ($\pm\infty$):** Esponente massimo (255 in precisione singola) e mantissa 0 [9, 12].
- **NaN (Not a Number):** Esponente massimo e mantissa diversa da zero; indica operazioni non valide come $0/0$ [9, 12].
- **Numeri Denormalizzati:** Usati per gestire l'underflow quando un numero è troppo piccolo per essere normalizzato. Hanno esponente 0 e mantissa non nulla [9, 11].

4 Codifica del Testo

4.1 Dal Carattere al Bit

Il testo è gestito dai calcolatori come una sequenza di caratteri, ciascuno dei quali deve essere mappato in una sequenza di bit attraverso una funzione di codifica [1, 2]. Una codifica che utilizza n bit permette di rappresentare un totale di 2^n simboli distinti [1, 2].

4.2 Lo Standard ASCII

L'**ASCII** (*American Standard Code for Information Interchange*) è lo standard storico per la codifica dell'alfabeto anglosassone [1, 2].

- **ASCII Standard:** Utilizza **7 bit** per rappresentare 128 caratteri, inclusi numeri, lettere maiuscole e minuscole, punteggiatura e caratteri di controllo [1-3].
- **ASCII Esteso:** Poiché i calcolatori lavorano su byte (8 bit), l'ottavo bit è stato utilizzato per raddoppiare i simboli disponibili (256 in totale) [1, 3]. Tuttavia, non esiste un unico standard esteso; diverse tabelle (come la **ISO 8859-1** per l'Europa Occidentale) associano simboli diversi (es. lettere accentate) ai valori superiori a 127, creando problemi di compatibilità tra sistemi diversi [1, 4, 5].

4.3 Unicode e UTF-8

Per superare i limiti dell'ASCII e rappresentare tutti gli alfabeti mondiali (come il cirillico, il cinese o l'arabo), è stato introdotto lo standard **Unicode** [6, 7].

- **Unicode:** Può codificare potenzialmente fino a 2^{32} simboli diversi [6, 7].
- **UTF-8:** È la codifica più diffusa per Unicode. Utilizza un numero variabile di byte (da 1 a 4) per ogni carattere ed è **retrocompatibile con l'ASCII**, poiché i primi 128 caratteri Unicode corrispondono a quelli ASCII e occupano un solo byte [6, 7].

4.4 Esempio di Codifica

Prendendo come esempio la parola “Ciao”, la sua rappresentazione in ASCII/UTF-8 segue la mappatura dei singoli caratteri in valori decimali, poi convertiti in bit [4]:

- **C:** 67 ($0x43$) $\rightarrow$ 01000011
- **i:** 105 ($0x69$) $\rightarrow$ 01101001
- **a:** 97 ($0x61$) $\rightarrow$ 01100001
- **o:** 111 ($0x6F$) $\rightarrow$ 01101111

5 Cenni alle Reti Logiche

5.1 Valori Logici ed Elettronica

I calcolatori moderni sono realizzati mediante circuiti elettronici digitali che operano su due livelli di tensione fondamentali [1].

- **Alto, asserito (1):** Associato alla tensione di alimentazione V_{dd} [1].
- **Basso, negato (0):** Associato alla massa (tensione nulla) [1].

Eventuali livelli di tensione intermedi sono considerati non significativi e assunti solo durante le fasi transitorie [1].

5.2 Tipologie di Reti Logiche

Le reti logiche sono circuiti che trasformano valori logici in ingresso in valori logici in uscita e si dividono in due categorie principali [1]:

- **Reti Combinatorie:** L'uscita in ogni istante dipende esclusivamente dai valori degli ingressi in quell'istante. Non possiedono memoria del passato [2].
- **Reti Sequenziali:** L'uscita dipende sia dagli ingressi attuali che dalla storia degli ingressi passati. Queste reti possiedono una memoria interna, definita **stato** della rete [2].

5.3 Algebra di Boole e Porte Logiche

Le funzioni logiche possono essere espresse in modo compatto tramite l'algebra di Boole, basata su tre operatori fondamentali implementati fisicamente dalle corrispondenti **porte logiche** [3, 4]:

1. **AND ($\cdot$):** Produce 1 solo se tutti gli operandi sono 1 [3].
2. **OR ($+$):** Produce 1 se almeno uno degli operandi è 1 [3].
3. **NOT ($\bar{A}$):** Inverte il valore logico dell'ingresso [5].

Sono di fondamentale importanza le **leggi di De Morgan**, che permettono di ricavare qualsiasi operatore logico partendo da porte di tipo NAND o NOR [6].

5.4 Circuiti Combinatori Fondamentali

Tra i blocchi logici più utilizzati nell'architettura di un processore si distinguono:

- **Decoder:** Un circuito con n ingressi e 2^n uscite; in base alla combinazione binaria in ingresso, viene attivata una sola delle uscite [4].
- **Multiplexer (Mux):** Un selettore che, in base a un segnale di controllo, determina quale tra diversi ingressi dati deve essere trasferito in uscita [4].

5.5 Elementi di Memoria

La capacità di memorizzare informazioni in una rete logica si ottiene attraverso la **retroazione** (o reazione), ovvero riportando l'uscita di una porta in ingresso ad altre porte dello stesso stadio [7, 8].

- **Latch S-R:** Elemento base con ingressi *Set* e *Reset*. Presenta lo svantaggio di uno stato "indecidibile" quando entrambi gli ingressi sono a 1 [8, 9].

- **Latch D (Gated):** Risolve l'ambiguità utilizzando un unico ingresso dati e un segnale di abilitazione chiamato **Clock** [9].
- **Flip-Flop:** Circuiti **edge-triggered** che caricano l'input solo sul fronte (salita o discesa) del clock, garantendo stabilità e prevenendo situazioni di corsa critica (*race condition*) [10, 11].
- **Registri:** Collezioni di flip-flop utilizzati per memorizzare parole di dati (*word*) all'interno del processore [10, 12].

6 Il Linguaggio Assembly

6.1 Il Gap tra Umano e Calcolatore

I calcolatori elettronici sono in grado di comprendere ed eseguire esclusivamente il proprio **linguaggio macchina**, ovvero una sequenza di cifre binarie (0 e 1) [1]. Al contrario, i programmatori utilizzano solitamente linguaggi ad **alto livello** (come C, C++, Java) che sono astratti e comprensibili dall'uomo, ma non direttamente dalla CPU [1].

L'**Assembly** rappresenta il punto di incontro: utilizza **codici mnemonici** (come `add`, `sub`, `lw`) al posto delle sequenze di bit, mantenendo una stretta corrispondenza con le istruzioni macchina [1]. Un programma chiamato **Assembler** ha il compito di tradurre questi codici mnemonici nella sequenza finale di bit [1].

6.2 Instruction Set Architecture (ISA)

L'**Instruction Set** è il vocabolario di istruzioni che una CPU è in grado di riconoscere [2]. Sebbene ogni processore abbia il proprio, le differenze tra di essi sono spesso paragonabili a inflessioni regionali di una stessa radice linguistica [2]. Le scelte di progettazione di un ISA si basano su criteri pratici: semplicità dei dispositivi, chiarezza delle applicazioni e velocità di esecuzione [3].

Le architetture si dividono principalmente in due categorie:

- **RISC (Reduced Instruction Set Computer):** Come il **MIPS** o il **RISC-V**, caratterizzate da istruzioni semplici, di lunghezza fissa e un numero elevato di registri [4]. L'accesso alla memoria avviene solo tramite istruzioni specifiche di *load* e *store* [4].
- **CISC (Complex Instruction Set Computer):** Come l'architettura **Intel x86**, con istruzioni di lunghezza variabile e operazioni che possono accedere direttamente alla memoria [4].
- **ARM:** Un'architettura "pragmatica" che mescola caratteristiche RISC con modalità di indirizzamento più complesse [4].

6.3 Operandi e Registri

A differenza dei linguaggi ad alto livello, dove le variabili sono potenzialmente infinite, in Assembly gli operandi delle istruzioni aritmetiche sono vincolati a risiedere nei **registri** [5].

- **Registri:** Locazioni di memoria interne al processore, estremamente veloci (accesso in un solo ciclo di clock) [5].
- **Principio di Progettazione:** “Minori sono le dimensioni, maggiore è la velocità” [6]. Limitare il numero di registri (es. 32 nel RISC-V) permette ai segnali elettrici di percorrere distanze minori, mantenendo alte le frequenze di clock [6].

6.4 Controllo del Flusso e Salti

Per implementare cicli (**while**, **for**) e selezioni (**if-else**), l'Assembly utilizza istruzioni di **salto condizionato** [7]. L'esecuzione di queste istruzioni dipende dal confronto tra valori contenuti nei registri o dallo stato di particolari **flag** (segnali logici) impostati da operazioni precedenti [7].

6.5 Principi di Progettazione dell'ISA

Nello sviluppo di linguaggi Assembly moderni come il RISC-V, si seguono tre principi fondamentali:

1. **La semplicità favorisce la regolarità:** Istruzioni uniformi semplificano l'hardware [8].
2. **Minori dimensioni, maggiore velocità:** Pochi registri rendono la CPU più rapida [6].
3. **Un buon progetto richiede buoni compromessi:** Ad esempio, l'uso di una lunghezza fissa per le istruzioni a scapito della densità del codice [9].

7 L'Architettura RISC-V

7.1 Caratteristiche Generali

Il RISC-V è un'architettura di tipo **RISC** (Reduced Instruction Set Computer) moderna e open-source, sviluppata all'Università di Berkeley nel 2010 [1]. Segue i principi di semplicità e regolarità per favorire l'efficienza dell'hardware e la velocità di esecuzione [1, 2].

7.2 I Registri nel RISC-V

A differenza della memoria RAM, i registri sono locazioni interne al processore estremamente rapide, il cui accesso avviene tipicamente in un solo ciclo di clock [3].

- **Quantità:** Il RISC-V dispone di **32 registri** general-purpose, numerati da **x0** a **x31** [4, 5].
- **Registro x0:** Ha la particolarità di essere cablato al valore costante **0**. Ogni tentativo di scrivervi viene ignorato, il che semplifica molte operazioni comuni [5, 6].
- **Dimensione:** In base all'implementazione (RV32I o RV64I), i registri possono essere a 32 o 64 bit [7].

7.3 Organizzazione della Memoria

La memoria è vista come un'ampia sequenza di bit organizzati in gruppi di 8 (**byte**), dove ciascun byte è associato a un indirizzo lineare progressivo [8].

- **Word e Double Word:** Nonostante l'indirizzamento al byte, il RISC-V opera solitamente su parole (*word*, 4 byte) o parole doppie (*double word*, 8 byte) [8].
- **Endianness:** Il RISC-V utilizza la convenzione **Little Endian**, in cui il byte meno significativo di una parola viene memorizzato all'indirizzo di memoria più basso [9].
- **Allineamento:** È possibile accedere a parole poste a indirizzi multipli dell'ampiezza della parola stessa (vincolo di allineamento), sebbene il RISC-V sia più flessibile di altre architetture in merito [10].

7.4 Trasferimento Dati: Load e Store

Le operazioni aritmetico-logiche in RISC-V avvengono esclusivamente tra registri [3, 8]. Per manipolare dati in memoria occorrono istruzioni di trasferimento:

- **Load (ld, lw, lb):** Preleva un dato dalla memoria e lo carica in un registro [8].
- **Store (sd, sw, sb):** Salva il contenuto di un registro in memoria [8].
- **Indirizzamento:** L'indirizzo di memoria viene calcolato sommando il contenuto di un **registro base** a uno **spiazzamento** (offset) costante [10, 11]. Esempio: `ld x9, 64(x22)` carica in x9 la doppia parola all'indirizzo contenuto in x22 aumentato di 64 [11].

7.5 Rappresentazione delle Istruzioni

Ogni istruzione RISC-V è codificata come un numero binario di **32 bit** [5]. La struttura dell'istruzione è suddivisa in campi specifici che ne definiscono il comportamento [4]:

1. **codop (opcode):** Codice operativo che identifica il tipo di istruzione.
2. **rs1 e rs2:** Registri sorgente (5 bit ciascuno, per indirizzare i 32 registri).
3. **rd:** Registro destinazione per il risultato.
4. **funz3 e funz7:** Campi aggiuntivi per specificare varianti della stessa operazione (es. somma vs sottrazione).

7.6 Pseudo-istruzioni

Per facilitare la scrittura del codice, l'Assembler supporta le **pseudo-istruzioni**, codici mnemonici che non esistono nel linguaggio macchina ma vengono tradotti in istruzioni reali [12]. Ad esempio, `mv x10, x11` (copia registro) viene tradotta internamente come `addi x10, x11, 0` [13].

8 Flusso di Controllo e Procedure

8.1 Istruzioni di Salto Condizionato

Per implementare costrutti di selezione come `if-else` e cicli (`while`, `for`), l'Assembly utilizza istruzioni di salto condizionato che alterano l'indirizzo della prossima istruzione da eseguire solo se una specifica condizione logica è verificata [1, 2]. Nel RISC-V, le istruzioni principali confrontano il contenuto di due registri [2].

- `beq rs1, rs2, Label`: Effettua il salto all'etichetta specificata se il valore in `rs1` è uguale a quello in `rs2` [2, 3].
- `bne rs1, rs2, Label`: Effettua il salto se i valori nei due registri sono diversi [2, 3].
- **Altri confronti**: Esistono istruzioni per verificare se un valore è minore di (`blt`) o maggiore/uguale (`bge`), sia per numeri con segno che senza segno (`bltu`, `bgeu`) [4].

Le sequenze di istruzioni comprese tra due salti condizionati sono denominate **blocchi di base**; l'identificazione di tali blocchi è una delle prime fasi compiute dal compilatore per tradurre i cicli dei linguaggi ad alto livello [4, 5].

8.2 Protocollo di Chiamata delle Procedure

L'utilità di un calcolatore dipende dalla possibilità di invocare procedure (o funzioni), viste come "scatole nere" che eseguono un compito specifico [6]. Per garantire l'interoperabilità tra diverse parti di un programma, viene definito un protocollo (o convenzione) di chiamata [6, 7].

1. **Passaggio parametri**: L'idea di base è utilizzare i registri, poiché sono il meccanismo più veloce; il RISC-V dedica i registri da `x10` a `x17` (`a0-a7`) per i parametri in ingresso e per i valori di ritorno [7-9].
2. **Salvataggio indirizzo di ritorno**: Il registro `x1` (`ra`, *return address*) memorizza l'indirizzo a cui tornare al termine della procedura [8, 10].
3. **Istruzioni di salto**: L'istruzione `jal` (*jump and link*) esegue il salto e salva simultaneamente l'indirizzo dell'istruzione successiva in `ra` [8, 11].
4. **Ritorno dal controllo**: Al termine, la procedura esegue `jalr x0, 0(x1)` per tornare al punto di partenza [8, 12].

8.3 Gestione dello Stack e Record di Attivazione

Se i registri disponibili non sono sufficienti per contenere tutti i parametri o le variabili locali, si ricorre allo **Stack** (pila) [13, 14].

- **Stack Pointer (sp)**: Il registro `x2` punta alla cima dello stack, che per convenzione cresce verso indirizzi di memoria decrescenti [13].
- **Prologo**: All'inizio di una procedura, il chiamante decrementa `sp` per allocare lo spazio necessario e salva i registri che devono essere preservati (operazione di *push*) [12, 13].

- **Epilogo:** Prima del ritorno, si ripristinano i valori originali dei registri caricandoli dallo stack (operazione di *pop*) e si incrementa **sp** per liberare la memoria [12, 15].
- **Frame Pointer (fp):** Alcuni programmi usano il registro **x8 (fp)** come base stabile per individuare le variabili locali tramite uno spiazzamento costante, poiché **sp** può variare durante l'esecuzione [16, 17].

8.4 Funzioni Foglia e Non Foglia

Si definisce **funzione foglia** una procedura che non chiama altre funzioni al suo interno [18, 19]. In ARM queste sono gestite in modo più semplice rispetto al RISC-V, poiché non è sempre obbligatorio salvare l'indirizzo di ritorno se non si effettuano ulteriori chiamate [19]. Al contrario, una **funzione non foglia** deve necessariamente salvare il proprio registro **ra** sullo stack nel prologo, altrimenti verrebbe sovrascritto dalla successiva istruzione **jal** [19, 20].

8.5 Organizzazione della Memoria

Il calcolatore organizza la memoria in segmenti logici distinti :

- **Stack:** Utilizzato per le variabili locali e i record di attivazione; procede verso indirizzi decrescenti .
- **Heap:** Destinato ai dati dinamici (allocati tramite **malloc** o **new**); procede verso indirizzi crescenti .
- **Dati Statici:** Locazioni per variabili globali, accessibili tramite il registro **x3 (gp, global pointer)** [14].
- **Testo (Code):** L'area di memoria che contiene le istruzioni macchina del programma in esecuzione .

9 L'Architettura Intel x86

9.1 Evoluzione e Caratteristiche CISC

L'architettura Intel rappresenta lo standard dominante per desktop, laptop e server [1]. A differenza del RISC-V, si tratta di un'architettura di tipo **CISC** (Complex Instruction Set Computer), caratterizzata da un set di istruzioni estremamente vasto e complesso, frutto di un'evoluzione iniziata negli anni '70 [1, 2]. Una caratteristica fondamentale è la **retrocompatibilità**: le moderne CPU a 64 bit sono ancora in grado di eseguire codice scritto per i vecchi processori a 8 o 16 bit, il che comporta una notevole complessità hardware [1, 2].

9.2 I Registri Intel

Il set di registri Intel a 64 bit è composto da 16 registri general-purpose, tutti identificati dal prefisso **%**.

- **Nomi classici:** **%rax, %rbx, %rcx, %rdx, %rsi, %rdi, %rbp, %rsp** [3, 4].

- **Nuovi registri:** Da `%r8` a `%r15` [3].
- **Struttura gerarchica:** Ogni registro a 64 bit include sottoregistri per la compatibilità con le ampiezze minori. Ad esempio, il registro `%rax` (64 bit) contiene `%eax` (32 bit), `%ax` (16 bit) e `%al` (8 bit) [3, 5].
- **Registri speciali:** `%rsp` funge da *stack pointer*, mentre `%rbp` è il *base pointer* (puntatore al record di attivazione) [3, 4].

9.3 Formato delle Istruzioni e Operandi

A differenza del RISC-V, le istruzioni Intel sono prevalentemente a **2 operandi**, dove il secondo operando funge anche da destinazione implicita (*sorgente, destinazione*) [6].

- **Flessibilità:** Le istruzioni aritmetico-logiche possono operare direttamente sulla memoria [7].
- **Vincolo memoria-memoria:** Non è consentito avere entrambi gli operandi in memoria in una singola istruzione; è necessario utilizzare un registro di appoggio [7, 8].
- **Suffissi di ampiezza:** L'ampiezza dei dati è specificata da un carattere finale nell'opcode: `b` (8 bit), `w` (16 bit), `l` (32 bit) e `q` (64 bit) [9].

9.4 Modalità di Indirizzamento

L'architettura Intel offre una modalità di indirizzamento molto potente chiamata **indiretto scalato**, che permette di calcolare l'indirizzo finale con la formula:

$$\text{Indirizzo} = \text{base} + (\text{indice} \times \text{scala}) + \text{spiazzamento}$$

dove la scala può assumere solo i valori 1, 2, 4 o 8 [8, 10]. Questa struttura è ideale per scorrere array di diversi tipi di dati con una singola istruzione [10].

9.5 Istruzioni Particolari

- **leaq (Load Effective Address):** Utilizza l'hardware del calcolo degli indirizzi per eseguire operazioni aritmetiche (come somme e shift combinati) senza accedere effettivamente alla memoria [11, 12].
- **inc / dec:** Sommano o sottraggono 1. Sono preferite alla `add $1` perché occupano meno byte nella codifica binaria dell'istruzione [13].

9.6 Convenzione di Chiamata (ABI)

Per garantire la compatibilità tra programmi e librerie, l'interfaccia binaria (*System V ABI*) stabilisce quanto segue:

- **Passaggio parametri:** I primi 6 argomenti interi o puntatori vengono passati nei registri `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8` e `%r9`. Gli argomenti successivi vengono caricati sullo **stack** [4, 6].

- **Valore di ritorno:** Viene inserito nel registro `%rax` [4, 6].
- **Registri preservati:** Il chiamato deve salvare e ripristinare `%rbp`, `%rbx` e `%r12-%r15` se intende utilizzarli [4, 6].

10 L'Architettura Intel x86

10.1 Evoluzione e Caratteristiche CISC

L'architettura Intel rappresenta lo standard dominante per desktop, laptop e server [1]. A differenza del RISC-V, si tratta di un'architettura di tipo **CISC** (Complex Instruction Set Computer), caratterizzata da un set di istruzioni estremamente vasto e complesso, frutto di un'evoluzione iniziata negli anni '70 [1, 2]. Una caratteristica fondamentale è la **retrocompatibilità**: le moderne CPU a 64 bit sono ancora in grado di eseguire codice scritto per i vecchi processori a 8 o 16 bit, il che comporta una notevole complessità hardware [1, 2].

10.2 I Registri Intel

Il set di registri Intel a 64 bit è composto da 16 registri general-purpose, tutti identificati dal prefisso `%`.

- **Nomi classici:** `%rax`, `%rbx`, `%rcx`, `%rdx`, `%rsi`, `%rdi`, `%rbp`, `%rsp` [3, 4].
- **Nuovi registri:** Da `%r8` a `%r15` [3].
- **Struttura gerarchica:** Ogni registro a 64 bit include sottoregistri per la compatibilità con le ampiezze minori. Ad esempio, il registro `%rax` (64 bit) contiene `%eax` (32 bit), `%ax` (16 bit) e `%al` (8 bit) [3, 5].
- **Registri speciali:** `%rsp` funge da *stack pointer*, mentre `%rbp` è il *base pointer* (puntatore al record di attivazione) [3, 4].

10.3 Formato delle Istruzioni e Operandi

A differenza del RISC-V, le istruzioni Intel sono prevalentemente a **2 operandi**, dove il secondo operando funge anche da destinazione implicita (*sorgente, destinazione*) [6].

- **Flessibilità:** Le istruzioni aritmetico-logiche possono operare direttamente sulla memoria [7].
- **Vincolo memoria-memoria:** Non è consentito avere entrambi gli operandi in memoria in una singola istruzione; è necessario utilizzare un registro di appoggio [7, 8].
- **Suffissi di ampiezza:** L'ampiezza dei dati è specificata da un carattere finale nell'opcode: `b` (8 bit), `w` (16 bit), `l` (32 bit) e `q` (64 bit) [9].

10.4 Modalità di Indirizzamento

L'architettura Intel offre una modalità di indirizzamento molto potente chiamata **indiretto scalato**, che permette di calcolare l'indirizzo finale con la formula:

$$\text{Indirizzo} = \text{base} + (\text{indice} \times \text{scala}) + \text{spiazzamento}$$

dove la scala può assumere solo i valori 1, 2, 4 o 8 [8, 10]. Questa struttura è ideale per scorrere array di diversi tipi di dati con una singola istruzione [10].

10.5 Istruzioni Particolari

- **leaq (Load Effective Address)**: Utilizza l'hardware del calcolo degli indirizzi per eseguire operazioni aritmetiche (come somme e shift combinati) senza accedere effettivamente alla memoria [11, 12].
- **inc / dec**: Sommano o sottraggono 1. Sono preferite alla **add \$1** perché occupano meno byte nella codifica binaria dell'istruzione [13].

10.6 Convenzione di Chiamata (ABI)

Per garantire la compatibilità tra programmi e librerie, l'interfaccia binaria (*System V ABI*) stabilisce quanto segue:

- **Passaggio parametri**: I primi 6 argomenti interi o puntatori vengono passati nei registri **%rdi**, **%rsi**, **%rdx**, **%rcx**, **%r8** e **%r9**. Gli argomenti successivi vengono caricati sullo **stack** [4, 6].
- **Valore di ritorno**: Viene inserito nel registro **%rax** [4, 6].
- **Registri preservati**: Il chiamato deve salvare e ripristinare **%rbp**, **%rbx** e **%r12-%r15** se intende utilizzarli [4, 6].

11 L'Architettura ARM

11.1 Origini e Caratteristiche

L'architettura ARM (*Advanced RISC Machine*), nata negli anni '80 come Acorn RISC Machine, è oggi la più diffusa nei sistemi embedded, smartphone e tablet [1, 2]. Si definisce un'architettura RISC "pragmatica" poiché, pur mantenendo i principi di efficienza tipici del RISC, introduce modalità di indirizzamento e istruzioni potenti che ricordano il mondo CISC per ottimizzare la densità del codice e il risparmio energetico [1, 2].

11.2 I Registri ARM

ARM dispone di **16 registri a 32 bit**, denominati da **r0** a **r15** [3].

- **r0-r12**: Registri general-purpose. Per convenzione ABI, **r0-r3** sono usati per il passaggio dei parametri e i valori di ritorno [4, 5].
- **r13 (sp)**: *Stack Pointer*, punta alla cima dello stack [3].

- **r14 (lr)**: *Link Register*, memorizza l'indirizzo di ritorno dalle subroutine (analogo a **ra** in RISC-V) [3].
- **r15 (pc)**: *Program Counter*. A differenza di molte altre architetture, il PC è accessibile direttamente come registro [3].
- **CPSR/APSR**: Registri di stato che contengono i flag (**N**egative, **Z**ero, **C**arry, **V**overflow) usati per l'esecuzione condizionale [3, 6].

11.3 Formato delle Istruzioni e Flessibilità

Le istruzioni ARM sono tipicamente a **3 argomenti** (*destinazione, sorgente1, sorgente2*) [5]. Una caratteristica distintiva è che il terzo argomento può essere un valore immediato o un registro sottoposto a un'operazione di **shift o rotazione** (es. LSL, LSR) integrata nell'istruzione stessa senza costi aggiuntivi in cicli di clock [5, 7, 8].

11.4 Esecuzione Condizionale

Quasi tutte le istruzioni ARM possono essere eseguite in modo condizionale aggiungendo un suffisso di due lettere al codice mnemonico [6].

- **eq** (equal), **ne** (not equal), **lt** (less than), **gt** (greater than), ecc [6, 9].
- Questo approccio permette di evitare molti salti condizionati, migliorando l'efficienza della pipeline [10].

11.5 Modalità di Indirizzamento e Write-back

ARM offre modalità di indirizzamento alla memoria molto avanzate (tramite **ldr** e **str**):

- **Base + Offset/Indice**: L'indirizzo è calcolato come **[base + offset]** [11].
- **Pre-indexing**: L'indirizzo viene aggiornato *prima* dell'accesso. Se seguito dal simbolo **!**, il valore aggiornato viene salvato nel registro base (**write-back**) [11, 12].
- **Post-indexing**: L'accesso avviene all'indirizzo contenuto nel registro base, che viene aggiornato *dopo* l'operazione [11, 13]. Questa modalità è estremamente utile per scorrere array in cicli **while** o **for** [14, 15].

11.6 Istruzioni Multiple (LDM e STM)

ARM permette di caricare o salvare **multipli registri** con una singola istruzione (**ldm** e **stm**) [16]. Queste sono fondamentali nel prologo e nell'epilogo delle funzioni per gestire lo stack in modo efficiente, permettendo di salvare l'intero stato dei registri e l'indirizzo di ritorno con una sola riga di codice [17, 18].

12 La Toolchain: dalla Sorgente all'Esecuibile

12.1 Il Processo di Traduzione

Poiché una CPU è in grado di comprendere ed eseguire esclusivamente il proprio **linguaggio macchina** (sequenze di bit), i programmi scritti in linguaggi ad alto livello o in Assembly devono essere trasformati attraverso una catena di strumenti software chiamata **toolchain**.

- **Compilatore:** Traduce il codice sorgente (es. C) in un file in linguaggio Assembly.
- **Assembler:** Trasforma il file Assembly in un **file oggetto** binario.
- **Linker:** Unisce diversi file oggetto e librerie in un unico **file eseguibile** finale.

12.2 Il Ruolo dell'Assembler

L'Assembler non si limita alla semplice traduzione uno-a-uno dei codici mnemonici in bit. Le sue funzioni principali includono:

- **Gestione delle Pseudo-istruzioni:** Traduce comandi simbolici utili al programmatore in istruzioni reali. Ad esempio, nel RISC-V:
 - `mv x10, x11` viene tradotto in `addi x10, x11, 0`.
 - `li x9, 123` (carica immediato) viene tradotto in `addi x9, x0, 123`.
 - `j Label` (salto incondizionato) viene tradotto in `jal x0, Label`.
- **Gestione delle Label:** Sostituisce i nomi simbolici dei salti con gli indirizzi binari corrispondenti (spesso calcolati come offset relativi al Program Counter).
- **Gestione dei Dati:** Converte i numeri (decimali o esadecimali) e le stringhe di testo in sequenze di bit.

12.3 Struttura del File Oggetto

Il file prodotto dall'Assembler è organizzato in segmenti logici distinti:

1. **Header:** Specifica la dimensione e la posizione degli altri segmenti.
2. **Text Segment:** Contiene le istruzioni in linguaggio macchina.
3. **Data Segment:** Contiene i dati statici (variabili globali) e le costanti.
4. **Tabella dei Simboli:** Elenca le etichette definite nel file che potrebbero essere usate da altri moduli (es. nomi di funzioni).
5. **Tabella di Rilocalizzazione:** Indica quali istruzioni dipendono da indirizzi assoluti che verranno stabiliti solo nella fase finale.

12.4 Il Linker e la Creazione dell'Eseguibile

Il **Linker** ha il compito di "cucire" insieme i vari moduli. Le sue fasi principali sono:

- **Risoluzione dei simboli:** Associa ogni riferimento a una funzione o variabile esterna alla sua effettiva definizione.
- **Calcolo degli indirizzi:** Determina la posizione finale di ogni istruzione e dato nella memoria del calcolatore, aggiornando i riferimenti nel codice.

Il risultato è un file eseguibile pronto per essere caricato in memoria dal **Loader** del Sistema Operativo.

12.5 Linking Statico e Dinamico

- **Linking Statico:** Tutte le librerie necessarie vengono incluse direttamente nel file eseguibile. Questo lo rende più grande ma autonomo.
- **Linking Dinamico:** Il programma contiene solo riferimenti a librerie esterne (es. `.dll` o `.so`). Il collegamento avviene solo quando il programma viene caricato (*non-lazy*) o quando la funzione viene effettivamente chiamata (*lazy linking*), risparmiando spazio in memoria.